지능형 DB 10주

서브쿼리

SQL 개요

데이터베이스에 저장된 데이터를 추출 또는 입력, 수정, 삭제하는 스크립트 언어

DML (Data Manipulation Language) – 주로 개발자가 사용

SELECT, INSERT, UPDATE, DELETE

DDL (Data Definition Language) – 주로 DBA(데이터베이스관리자)가 사용

CREATE, ALTER, DROP

DCL (Data Control Language) - 주로 DBA(데이터베이스관리자)가 사용

GRANT, REVOKE

ANY, ALL

ANY : ANY 이후에 나오는 조건 중 하나라도 만족하면 결과를 반환한다.

> ANY의 경우 MIN과 같은 의미가 된다.

조건	결과	설명
1000 > ANY(500, 1000, 2000)	Y	1000보다 작은 500 존재
1000 = ANY(500, 1000, 2000)	Y	1000같은 값 존재
2000 < ANY(500, 1000, 2000)	N	2000보다 큰 값 부존재

ANY, ALL

급여가 1000, 2000, 3000 중 하나라도 큰 사원명, 급여를 출력하시오.

다르게 해석하면 급여가 1000보다 크기만 하면 실행한다.

```
SELECT *
FROM emp
WHERE sal > ANY (1000, 2000, 3000);
```

보충 문제 8를 풀어보자.

ENAME	SAL	DEPTNO
KING	5000	10
FORD	3000	20
JONES	2975	20
BLAKE	2850	30
CLARK	2450	10
ALLEN	1600	30
TURNER	1500	30

ANY, ALL

ALL : ALL 이후에 나오는 조건 모두 만족하면 결과를 반환한다.

> ALL의 경우 MAX와 같은 의미가 된다.

조건	결과	설명
1000 > ALL(500, 1000, 2000)	N	1000이 all 이하 모든 값보다 커야 만족
1000 = ALL(500, 1000, 2000)	N	1000이 all 이하 모든 값과 같아야 만족
400 < ALL(500, 1000, 2000)	Y	400이 all 이하 모든 값보다 작아야 만족

ANY, ALL

급여가 1000, 2000, 3000 중 가장 큰 값 보다 큰 사원명, 급여를 출력하시오.

다르게 해석하면 급여가 3000보다 크기만 하면 실행한다.

```
SELECT ename, sal
FROM emp
WHERE sal > all (1000, 2000, 3000);
```

보충문제 9를 풀어보자.

Having절의 서브 쿼리

직업과 직업별 합산 금액을 출력하는데,

직업이 'CLERK'인 사원들의 합산 급여보다 높은 금액을 출력하시오.

```
SELECT job, sum(sal)
FROM emp
GROUP by job
```

Having절의 서브 쿼리

```
SELECT sum(sal)
FROM emp
WHERE job = 'CLERK'
```

'그룹 함수로 검색 조건'을 이용할 시 'WHERE절'을 사용할 수 없다.

반드시 'HAVING절'을 이용해야 한다.

Having절의 서브 쿼리

잘못 작성된 예

```
SELECT job, sum(sal)
FROM emp
WHERE sum(sal) > (SELECT sum(sal)
                  FROM emp
                  WHERE job = 'CLERK')
GROUP by job;
```

Having절의 서브 쿼리

앞장의 오류를 수정한 예

```
SELECT job, sum(sal)
FROM emp
GROUP by job
HAVING sum(sal) > (SELECT sum(sal)
                   FROM emp
                   WHERE job = 'CLERK');
```

JOB	SUM(SAL)
SALESMAN	5600
ANALYST	6000
MANAGER	8275
PRESIDENT	5000

서브 쿼리 유형

유형	서브쿼리 가능	명칭
SELECT	가능	스칼라 서브 쿼리
FROM	가능	인 라인 뷰
WHERE	가능	서브 쿼리
GROUP by	불가능	
HAVING	가능	
ORDER by	가능	

FROM절의 서브 쿼리 (In Line View)

사원명과 급여, 순위를 출력하는데,

급여 순위가 1위인 사원을 출력하시오.

```
SELECT ename, sal, rank() over (order by sal desc)
FROM emp

WHERE rank() over (order by sal desc) = 1;
```

FROM절의 서브 쿼리 (In Line View)

작성된 쿼리는 오류를 갖고 있다.

'WHERE절'에는 '분석함수(RANK() OVER)'를 사용할 수 없다.

'FROM절'에 서브 쿼리를 사용하여 먼저 수행하게 하고 그 결과값을 하나의 집합으로 만들어야 수행할 수 있다.

FROM절의 서브 쿼리 (In Line View)

```
SELECT v.ename, v.sal, v.ranking
FROM (select ename, sal, rank() over (order by sal desc) ranking from emp) v
WHERE v.ranking = 1;
```

ENAME	SAL	RANKING
KING	5000	1

아래의 해법은 무엇이 문제일까?

```
SELECT ROWNUM, ename, sal
FROM emp
ORDER by sal DESC; /* 랭킹이 무작위 순서로 출력 */
```

SELECT절의 서브 쿼리

직업이 'SALESMAN'인 사원의 이름과 급여를 출력하는데,

직업이 'SALESMAN'인 사원의 최대 급여와 최소급여를 같이 출력하시오.

```
SELECT ename, sal, max(sal), min(sal)
FROM emp
WHERE job = 'SALESMAN';
/* GROUP by job Having job = 'SALESMAN'; */
```

SELECT절의 서브 쿼리

```
SELECT ename, sal, (select max(sal) from emp where job = 'SALESMAN') 최대,
                   (select min(sal) from emp where job = 'SALESMAN') 최소
FROM emp
WHERE job = 'SALESMAN';
```

SELECT절의 쿼리는 해당 사원 수 만큼 수행되지만 주기억장치에 max, min값은 한번 계산된 후 그 값을 계속 이용한다.

이러한 방법을 '서브 쿼리 캐싱'이라고 한다.

ENAME	SAL	최대	최소
MARTIN	1250	1600	1250
ALLEN	1600	1600	1250
TURNER	1500	1600	1250
WARD	1250	1600	1250

WITH ~ AS 절 이용하기(현업 다수 사용)

동일한 SQL이 반복되어 사용될 때 성능 향상의 목적이다.

현업에서는 매우 많이 사용한다.

쿼리 성능에 매우 효과적이다.

조인을 회피할 수 있다.

임시 테이블을 주기억장치에 생성한 후 그 테이블을 이용하여 쿼리를 수행한다.

WITH ~ AS 절 이용하기

직업과 직업별 합산급여를 출력하되

직업별 합산급여가 평균 급여액 보다 많은 데이터만 출력하시오.

```
SELECT job, sum(sal)
FROM emp
GROUP by job
```

WITH ~ AS 절 이용하기

```
SELECT job, sum(sal)
FROM emp
WHERE sum(sal) > ( select avg(sum(sal))
                   from emp
                   group by job);
```

WITH ~ AS 절 이용하기

합쳐서 하나의 쿼리로 만들면..

```
SELECT job, sum(sal)
FROM emp
GROUP by job
HAVING sum(sal) > ( select avg(sum(sal))
                    from emp
                    group by job);
```

JOB	SUM(SAL)
SALESMAN	5600
ANALYST	6000
MANAGER	8275

WITH ~ AS 절 이용하기

WITH절을 이용하면 WITH절은 컴파일하여 결과값을 주기억장치에 임시로 저장

```
WITH TOTAL_JOB AS (SELECT job, sum(sal) total
                   FROM emp
                   GROUP by job)
SELECT job, total
FROM TOAL_JOB
WHERE total > ( select avg(total)
                from TOTAL_JOB);
```

JOB	TOTAL
MANAGER	8275
ANALYST	6000
SALESMAN	5600

Subquery Factoring(임시 테이블 생성 모음)

서브 쿼리 2개 이상 WITH ~ AS절로 생성

생성된 임시 테이블을 참조하여 원하는 데이터를 추출하는 방법

WITH절을 사용하면 특정 서브 쿼리문의 컬럼을 다른 서브 쿼리문에서 참조하는 것이 가능함

하나의 WITH절에 필요한 여러 쿼리를 생성하여 주기억장치에 올려 놓고 사용함

WITH 임시테이블명1 AS (SELECT ~), 임시테이블명2 AS (SELECT ~) ~~

Subquery Factoring(임시 테이블 생성 모음)

직업별 합산 급여액의 평균 액수에 3000을 더한값 보다 더큰

부서번호 별 합산 급여를 출력하시오.

```
SELECT job, sum(sal) total
FROM emp
Group by job
```

위의 SQL문을 WITH ~ AS로 전환하여 임시테이블 형성

Subquery Factoring(임시 테이블 생성 모음)

```
SELECT deptno, sum(sal)
FROM emp
GROUP by deptno /* 부서번호별 */
조건
HAVING sum(sal) > ( select avg(        ) + 3000
FROM [                          ]
```

Subquery Factoring(임시 테이블 생성 모음)

```
SELECT deptno, sum(sal)
FROM emp
GROUP by deptno /* 부서번호별 */
/* WITH ~ AS */
위의 group by의 조건
HAVING sum(sal) > ( select avg(total) + 3000
FROM 임시테이블명1)
```

Subquery Factoring(임시 테이블 생성 모음)

```
WITH SUM_SAL AS ( select job, sum(sal) total from emp
                  group by job),
     DEPT_SUM AS (select deptno, sum(sal) total from emp
                  group by deptno
                  having sum(sal) > ( select avg(total) + 3000
                                      from SUM_SAL) )
SELECT deptno, total
FROM DEPT_SUM;
```

DEPTNO	TOTAL
30	12400
10	8750
20	10875

Subquery Factoring(임시 테이블 생성 모음)

혹시 in-line view를 사용하면 해결되지 않을까? (from절에 서브쿼리)

SELECT

FROM 서브쿼리1, 서브쿼리2;

Subquery Factoring(임시 테이블 생성 모음)

```
SELECT deptno, sum(sal)
FROM ( select job, sum(sal) total from emp
          group by job) SUM_SAL,
       (select deptno, sum(sal) total from emp
        group by deptno
        having sum(sal) > (select avg(total) + 3000 from SUM_SAL)) DEPT_SUM;
```

왜 에러일까?

Subquery Factoring(임시 테이블 생성 모음)

```
SELECT deptno, sum(sal)
FROM ( select job, sum(sal) total from emp
        group by job) SUM_SAL,
      (select deptno, sum(sal) total from emp
       group by deptno
       having sum(sal) > (select avg(total) + 3000 from SUM_SAL)) DEPT_SUM;
```

같은 FROM절의 테이블 별칭은 다른 곳에서 참조하여 사용할 수 없다.

INSERT문 (데이터 입력)

INSERT INTO 테이블명 (어트리뷰트명1, ~, ~, ~) VALUES (데이터1, ~, ~, ~);

사원번호 : 2812

사원명 : 'JACK'

급여 : 3500

입사일 : (19)98년 6월 5일

직업 : 'ANALYST'

부서번호 : 20

INSERT문 (데이터 입력)

INSERT INTO emp (empno, ename, sal, hiredate, job, deptno)

VALUES (2812, 'JACK', 3500, '98/06/05', 'ANALYST', 20);

위의 문제점은? 입력은 정상적으로 이루어진다. (묵시적 형 변환하여 입력)

날짜를 유의해서 보자!

/* '98/06/05' */ 문자형이 아닌 날짜형으로 입력해야 한다.

to_date('98/06/05', 'RR/MM/DD')

INSERT문 (데이터 입력)

입력해야 할 항목에 값이 없는 경우는 NULL처리한다.

보충문제 1번을 풀어보세요.

보충문제 2번을 풀어보세요.

보충문제 3번을 풀어보세요.

주의 사항 보충문제 3번의 경우 싱글 쿼트 2개를 스페이스 없이 바로 붙여 사용!

*** 주의 : insert, update, delete문의 실행 결과는 주기억장치에 기록된다 !!!

UPDATE문 (데이터 수정)

UPDATE 테이블명

SET 데이터 수정

WHERE 조건 ;

'SCOTT'의 급여를 4100으로 변경해보자.

UPDATE문 (데이터 수정)

```
UPDATE emp
SET sal = 4100
WHERE ename = 'SCOTT';
```

모든 사원의 급여를 NULL로 변경하려면?

```
UPDATE emp SET sal = NULL; /*실제 실행하지는 말것! */
```

DELETE문 (데이터 삭제)

실제 업계에서는 거의 사용하지 않는다.

DELETE FROM 테이블명

WHERE 해당 조건;

행 전체를 삭제한다. /* 개별 데이터가 아닌 */

회사에서 'YOON'이 퇴사를 했다면

DELETE FROM emp

WHERE ename = 'YOON'; /* 'YOON'의 모든 데이터 삭제 */